RWorksheet_camasa#4c

April Marie

2025-01-04

library(ggplot2) library(dplyr)

#1. #1a. How to Import a CSV File

```
mpgdata <- read.csv("mpg.csv")
```

#b.

```
str(mpgdata)
```

```
## 'data.frame':    234 obs. of  12 variables:
##  $ X           : int  1 2 3 4 5 6 7 8 9 10 ...
##  $ manufacturer: chr  "audi" "audi" "audi" "audi" ...
##  $ model       : chr  "a4" "a4" "a4" "a4" ...
##  $ displ       : num  1.8 1.8 2 2 2.8 2.8 3.1 1.8 1.8 2 ...
##  $ year        : int  1999 1999 2008 2008 1999 1999 2008 1999 1999 2008 ...
##  $ cyl         : int  4 4 4 4 6 6 6 4 4 4 ...
##  $ trans       : chr  "auto(l5)" "manual(m5)" "manual(m6)" "auto(av)" ...
##  $ drv         : chr  "f" "f" "f" "f" ...
##  $ cty         : int  18 21 20 21 16 18 18 18 16 20 ...
##  $ hwy         : int  29 29 31 30 26 26 27 26 25 28 ...
##  $ fl          : chr  "p" "p" "p" "p" ...
##  $ class       : chr  "compact" "compact" "compact" "compact" ...
```

2. Which manufacturer has the most models in this data set? Which model has the most variations?

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
manufacturer_count <- mpgdata %>%
  group_by(manufacturer) %>%
  summarise(model_count = n_distinct(model)) %>%
  arrange(desc(model_count))
most_models_manufacturer <- manufacturer_count[1, ]
model_variation_count <- mpgdata %>%
  group_by(model) %>%
  summarise(variation_count = n()) %>%
```

```r
  arrange(desc(variation_count))
most_variations_model <- model_variation_count[1, ]
most_models_manufacturer
```

```
## # A tibble: 1 x 2
##   manufacturer model_count
##   <chr>              <int>
## 1 toyota                 6
```

```r
most_variations_model
```

```
## # A tibble: 1 x 2
##   model       variation_count
##   <chr>                 <int>
## 1 caravan 2wd              11
```

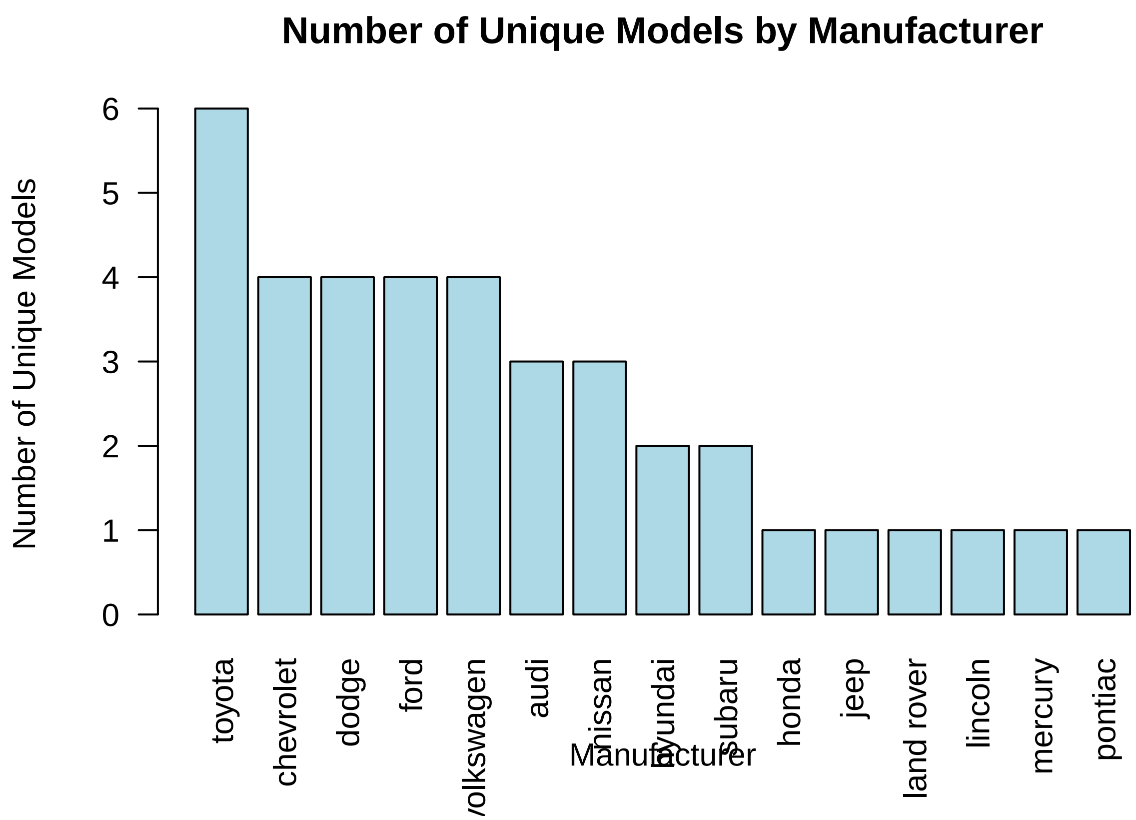
A.

```r
unique_models <- mpgdata %>%
  group_by(manufacturer) %>%
  summarise(unique_models = list(unique(model))) %>%
  arrange(manufacturer)
print(unique_models)
```

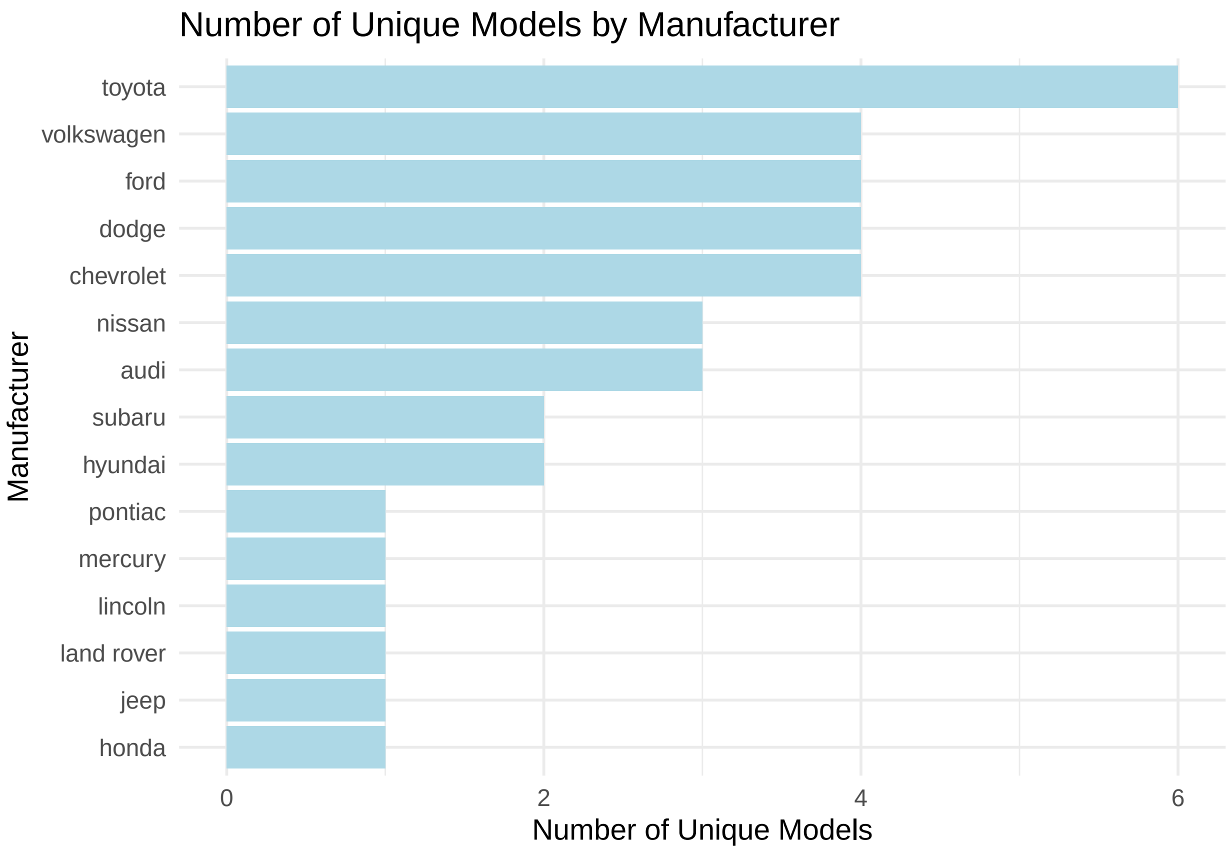
```
## # A tibble: 15 x 2
##    manufacturer unique_models
##    <chr>        <list>
##  1 audi         <chr [3]>
##  2 chevrolet    <chr [4]>
##  3 dodge        <chr [4]>
##  4 ford         <chr [4]>
##  5 honda        <chr [1]>
##  6 hyundai      <chr [2]>
##  7 jeep         <chr [1]>
##  8 land rover   <chr [1]>
##  9 lincoln      <chr [1]>
## 10 mercury      <chr [1]>
## 11 nissan       <chr [3]>
## 12 pontiac      <chr [1]>
## 13 subaru       <chr [2]>
## 14 toyota       <chr [6]>
## 15 volkswagen   <chr [4]>
```

B.

```r
model_counts <- mpgdata %>%
  group_by(manufacturer) %>%
  summarise(unique_model_count = n_distinct(model)) %>%
  arrange(desc(unique_model_count))
barplot(model_counts$unique_model_count,
        names.arg = model_counts$manufacturer,
        las = 2,
        col = "lightblue",
        main = "Number of Unique Models by Manufacturer",
        xlab = "Manufacturer",
        ylab = "Number of Unique Models")
library(ggplot2)
```




```r
ggplot(model_counts, aes(x = reorder(manufacturer, unique_model_count), y = unique_model_count)) +
  geom_bar(stat = "identity", fill = "lightblue") +
  coord_flip() +
  labs(title = "Number of Unique Models by Manufacturer",
       x = "Manufacturer",
       y = "Number of Unique Models") +
  theme_minimal()
```



2.

Same dataset will be used. You are going to show the relationship of the modeland the manufacturer.

a. What does ggplot(mpg, aes(model, manufacturer)) + geom_point() show?

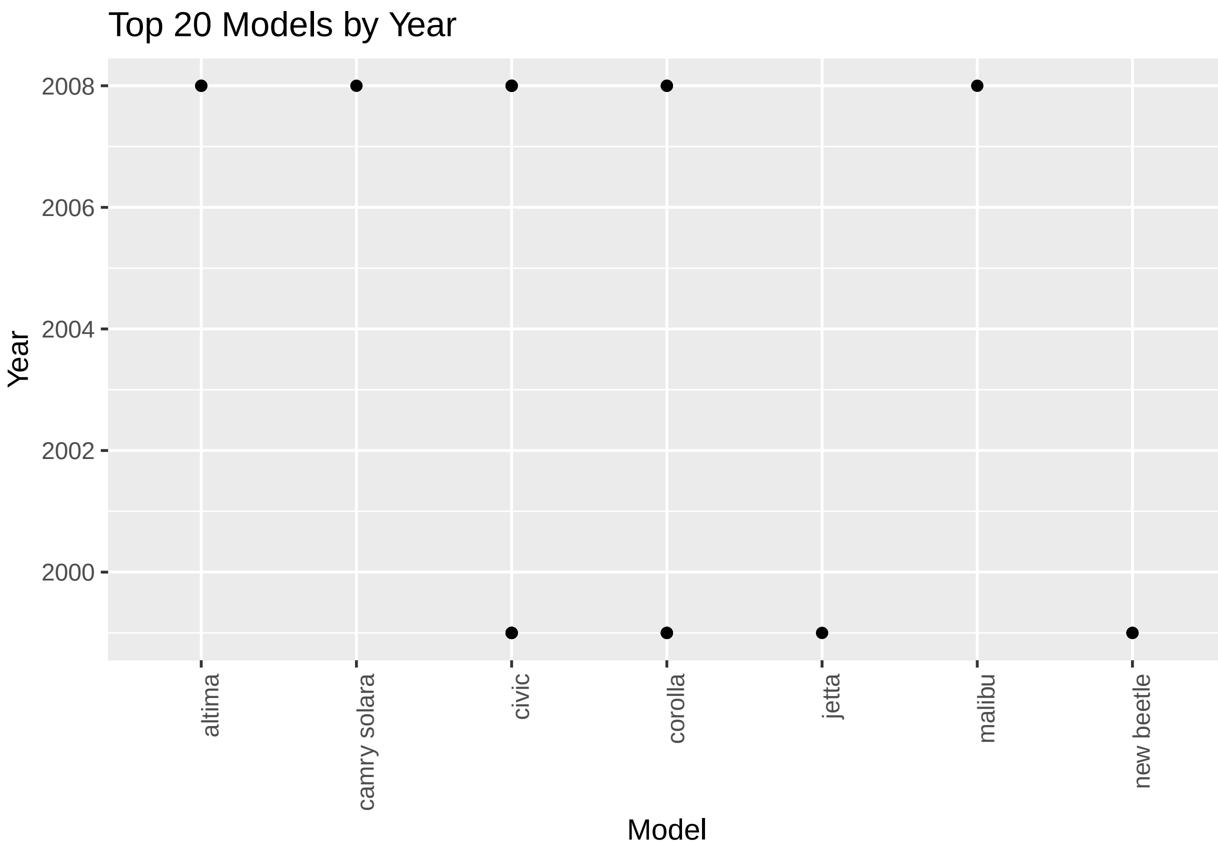
-the x-axis represents the model, and the y-axis represents the manufacturer. Since both variables are categorical, the plot will display points for each model-manufacturer pair. This can result in many overlapping points, making it difficult to identify any clear patterns or relationships. As a result, the plot may look cluttered and may not effectively convey the connection between these variables.

b. For you, is it useful? If not, how could you modify the data to make it more informative?

-As it is, the plot does not effectively reveal the relationship between the model and manufacturer due to the overplotting of points and the lack of numerical or continuous variables that could provide further insights.

3. Plot the model and the year using ggplot(). Use only the top 20 observations. Write the codes and its results.

```
library(ggplot2)
library(dplyr)
top_20_mpg <- mpgdata %>%
  arrange(desc(cty)) %>%
  head(20)
ggplot(top_20_mpg, aes(x = model, y = year)) +
  geom_point() +
  labs(title = "Top 20 Models by Year", x = "Model", y = "Year") +
  theme(axis.text.x = element_text(angle = 90, hjust = 1))
```



4. Using the pipe (%>%), group the model and get the number of cars per model. Show codes and its result.

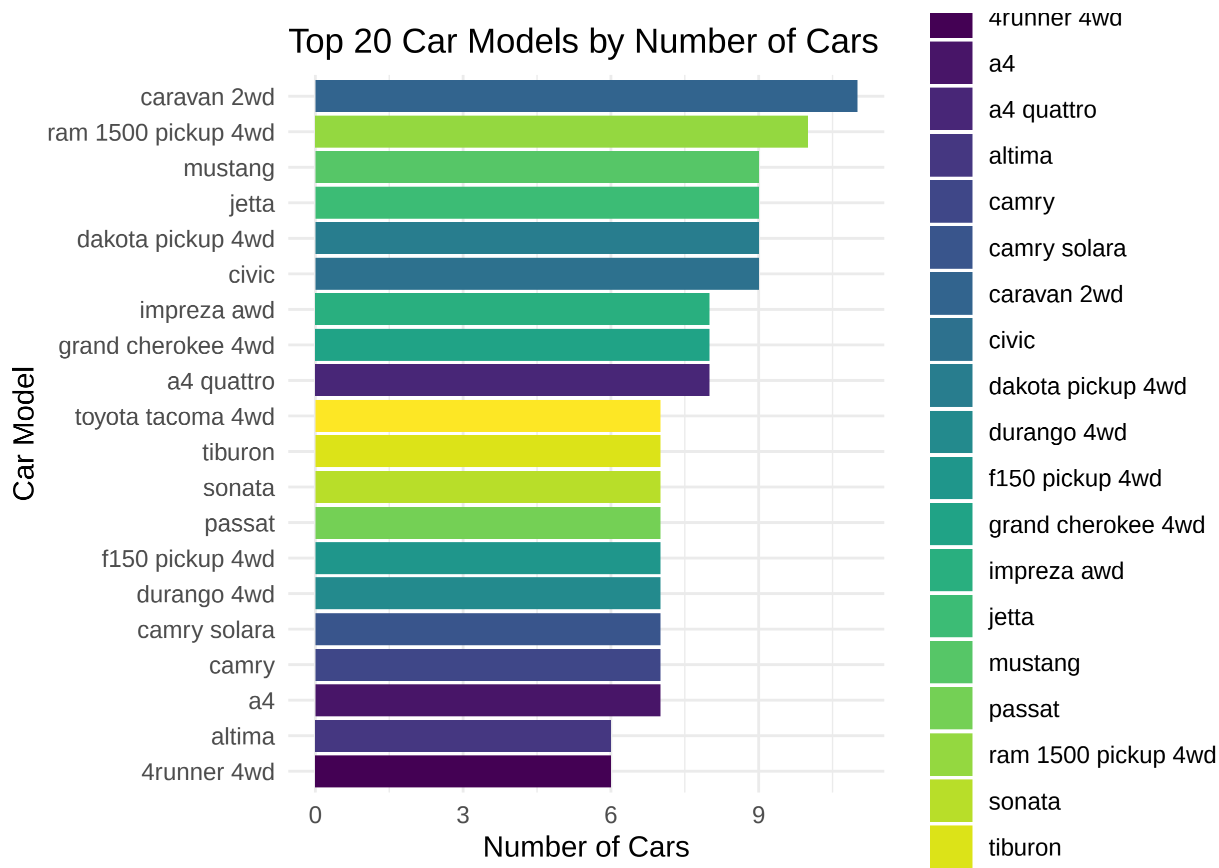
```
model_counts <- mpg %>%
  group_by(model) %>%
  summarise(number_of_cars = n()) %>%
  arrange(desc(number_of_cars))
print(model_counts)
```

```
## # A tibble: 38 x 2
##    model               number_of_cars
##    <chr>                        <int>
##  1 caravan 2wd                     11
##  2 ram 1500 pickup 4wd             10
##  3 civic                            9
##  4 dakota pickup 4wd                9
##  5 jetta                            9
##  6 mustang                          9
##  7 a4 quattro                       8
##  8 grand cherokee 4wd               8
##  9 impreza awd                      8
## 10 a4                               7
## # i 28 more rows
```

A.

```
top_models <- mpgdata %>%
  group_by(model) %>%
  summarise(number_of_cars = n()) %>%
```

```
  arrange(desc(number_of_cars)) %>%
  slice_head(n = 20)
ggplot(top_models, aes(x = reorder(model, number_of_cars), y = number_of_cars, fill = model)) +
  geom_bar(stat = "identity") +
  coord_flip() +
  labs(title = "Top 20 Car Models by Number of Cars",
       x = "Car Model",
       y = "Number of Cars",
       fill = "Model") +
  theme_minimal() +
  scale_fill_viridis_d()
```



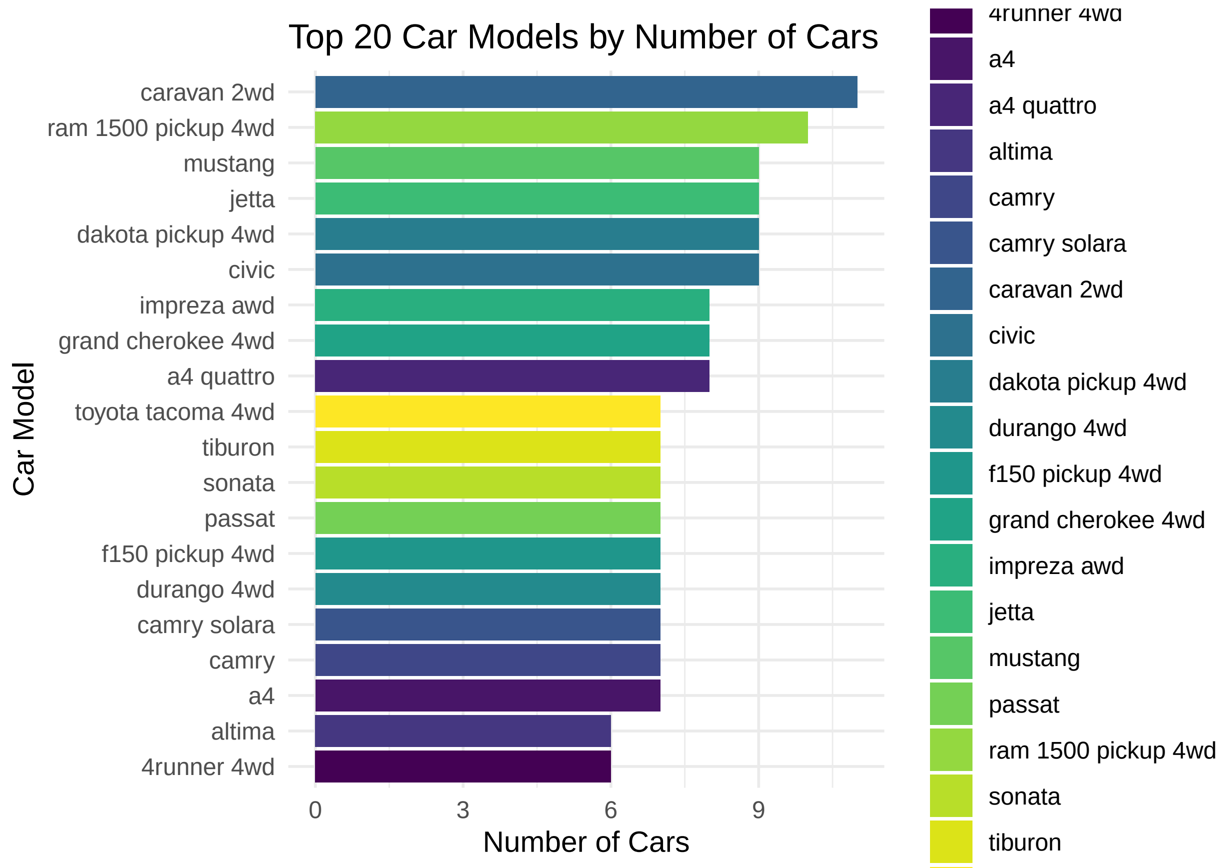
b. Plot using the geom_bar() + coord_flip() just like what is shown below. Show codes and its result.

```
library(viridis)
```

```
## Loading required package: viridisLite
```

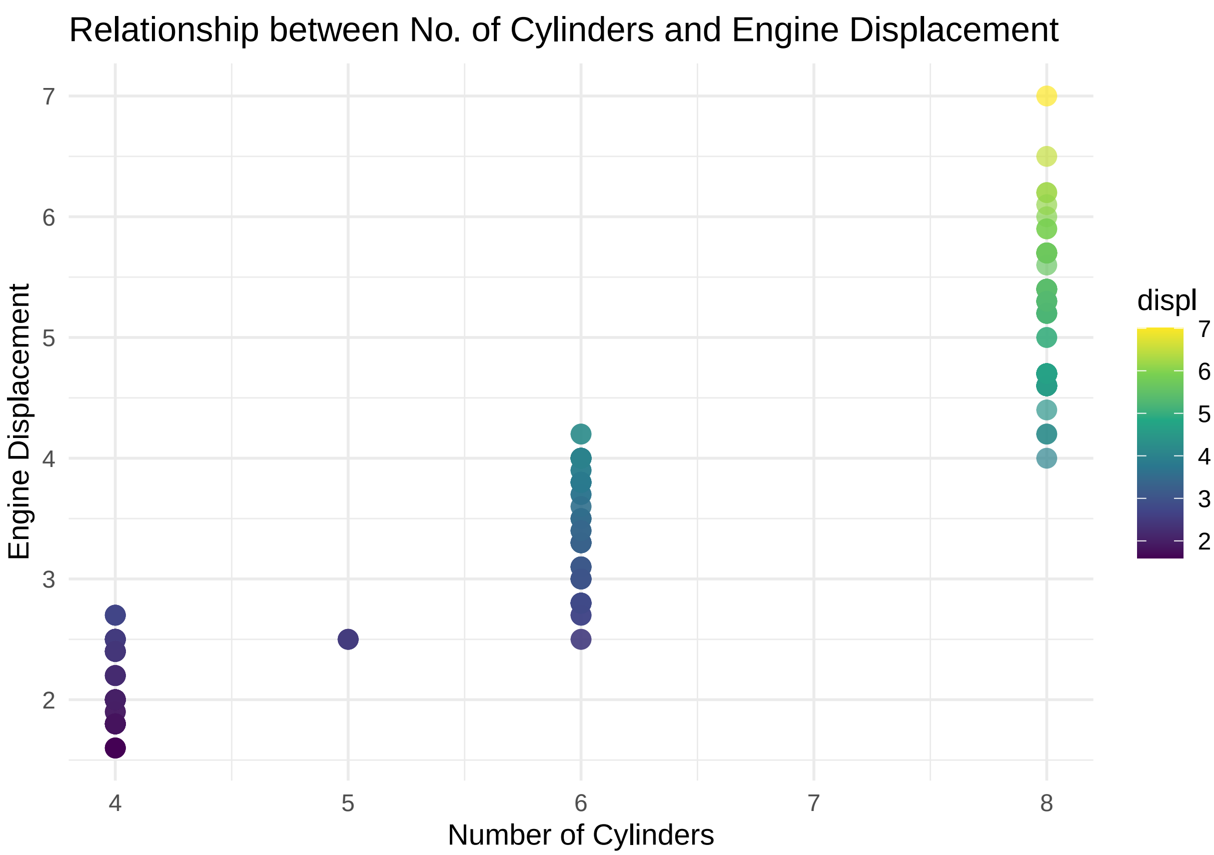
```
library(viridisLite)
top_models <- mpgdata %>%
  group_by(model) %>%
  summarise(number_of_cars = n()) %>%
  arrange(desc(number_of_cars)) %>%
  slice_head(n = 20)
ggplot(top_models, aes(x = reorder(model, number_of_cars), y = number_of_cars, fill = model)) +
  geom_bar(stat = "identity") +
  coord_flip() +
```

```r
  labs(title = "Top 20 Car Models by Number of Cars",
       x = "Car Model",
       y = "Number of Cars") +
  theme_minimal() +
  scale_fill_viridis_d()
```



5.

```r
ggplot(mpgdata, aes(x = cyl, y = displ, color = displ)) +
  geom_point(size = 3, alpha = 0.7) +
  labs(title = "Relationship between No. of Cylinders and Engine Displacement",
       x = "Number of Cylinders",
       y = "Engine Displacement") +
  theme_minimal() +
  scale_color_viridis_c()
```


Relationship between No. of Cylinders and Engine Displacement
Engine Displacement
Number of Cylinders
7
6
5
4
3
2
4
5
6
7
8
displ
7
6
5
4
3
2